

强化学习原理：概论

郭宪

2025.02.18

人工智能学院

College of Artificial Intelligence



南开大学
Nankai University



关于本门课



1. 强化学习概述及多臂赌博机

第一次实验：井字游戏和多臂赌博机

2. 马尔科夫决策过程及数学基础

3. 从动态规划到强化学习

第二次实验：MDP、策略迭代、值迭代

4. 表格型强化学习：蒙特卡洛和时间差分

第三次实验：蒙特卡洛方法及时间差分方法

5. 函数逼近方法及深度学习框架介绍

第四次实验：ModelArts 实验

6. 基于函数逼近的强化学习方法

第五次实验：深度强化学习DQN

7. 最新文献汇报

8. 最新文献汇报

9. 基于策略梯度的强化学习方法

第六次实验：基于策略梯度的方法

10. 基于置信域的强化学习方法

第七次实验：基于PPO的方法

11. 基于确定性策略的强化学习方法

第八次实验：基于DDPG的方法



关于本门课

12. 博弈论与多智能体强化学习方法

13. 多智能体强化学习方法

14. 最新文献汇报一

15. 最新文献汇报二

16. 最新文献汇报三及总复习

参考材料：

R.S Sutton, A.G Barto, Reinforcement Learning: An Introduction

<https://github.com/ShangtongZhang/reinforcement-learning-an-introduction>

郭宪, 方勇纯, 深入浅出强化学习：原理入门

郭宪, 宋俊潇, 方勇纯, 深入浅出强化学习：编程实战

https://github.com/Teacher-Guo/RL_code

各种公开发表的论文

助教：吴辉

助教邮箱：1120240305@mail.nankai.edu.cn

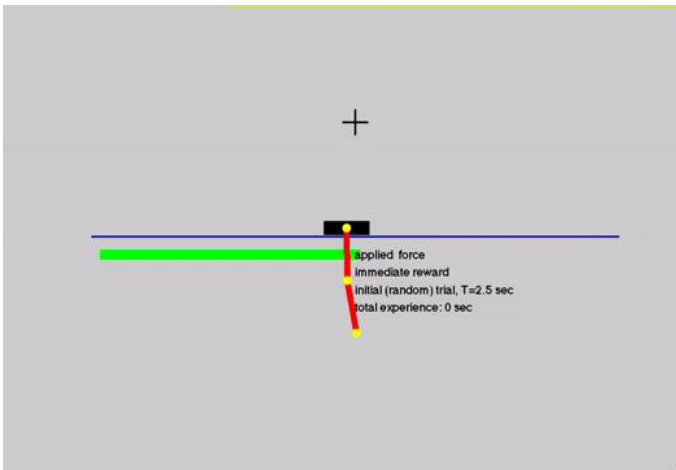


强化学习概述

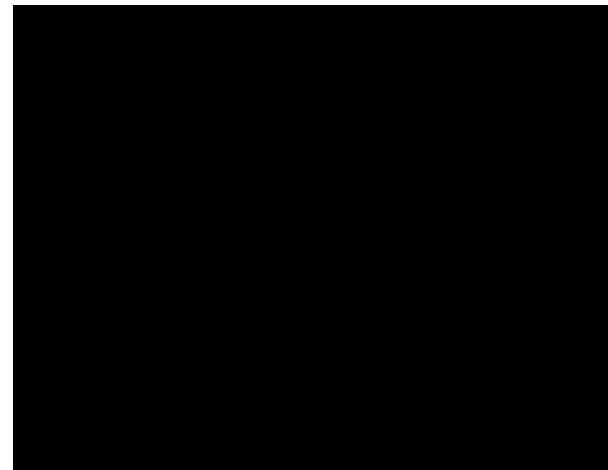


1. 强化学习能解决的问题
2. 强化学习与其他学科的联系
3. 强化学习如何去解决问题
4. 强化学习算法历史
5. 强化学习的分类
6. 强化学习的发展趋势
7. 强化学习课程路线图
8. 课题组的一些工作

1. 强化学习能解决的问题



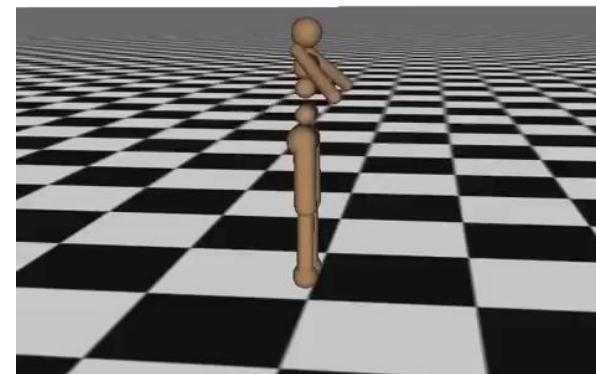
非线性控制



Iteration 0 视频游戏



AlphaGo

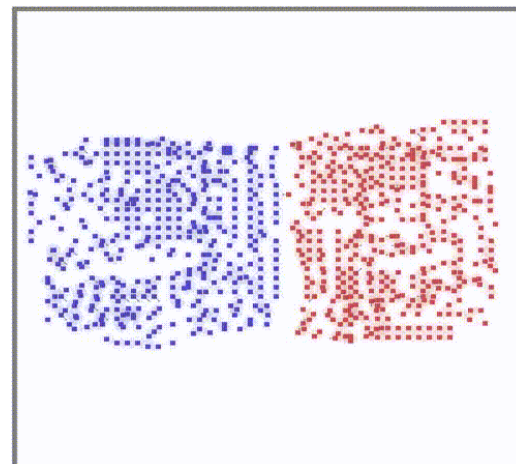


机器人，走路，站立

1. 强化学习能解决的问题



启元世界，多智能体协作（星际）



海量智能体（上海交通大学）

1. 强化学习能解决的问题

星际争霸2

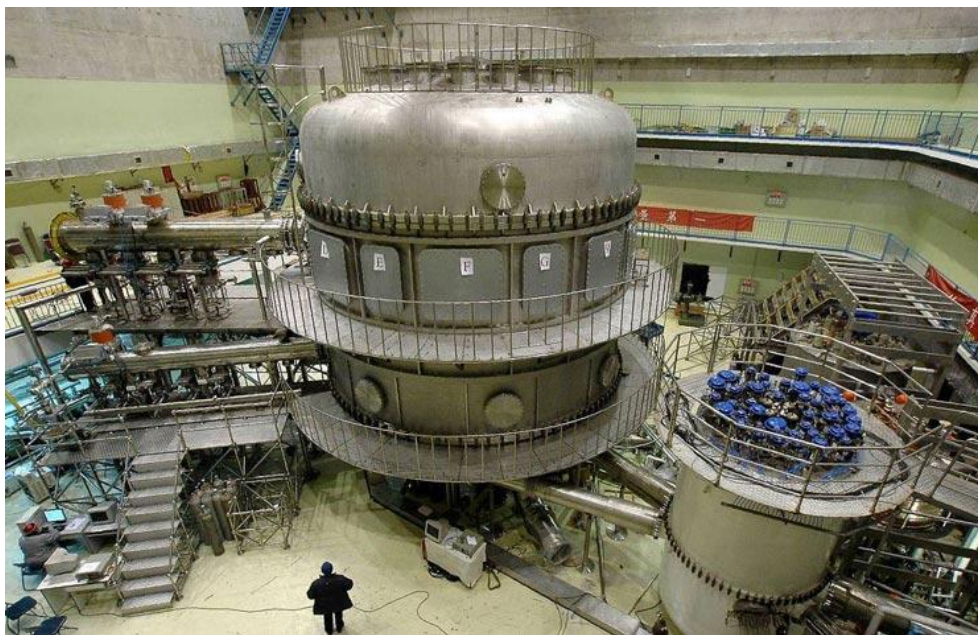
- 由暴雪娱乐开发的即时战略游戏 (RTS game)
- 造农民采矿、造建筑、造兵、攀科技, 最后派兵拆光对手的建筑
- 正规比赛中的设定是1v1, 每个玩家选择一个种族 (人类Terran, 神族 Protoss, 虫族 Zerg)进行对抗
- 目标: 优先摧毁对方所有建筑



星际争霸实战场面

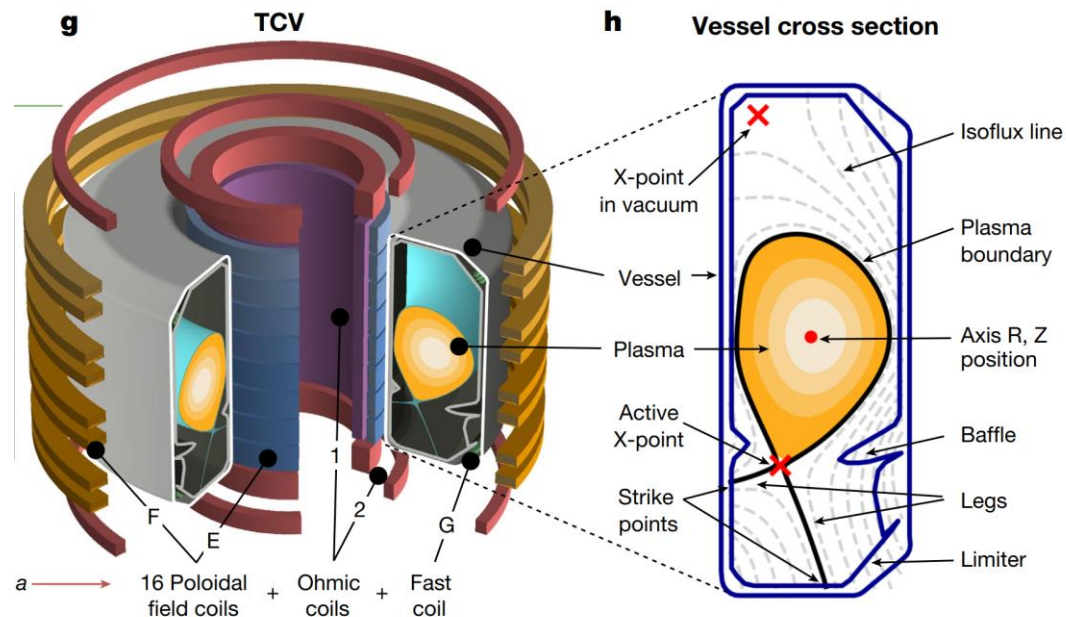
1. 强化学习能解决的问题

可控核聚变:



托马克装置

➤ 用强大的**电磁力约束**核聚时超高温等离子体的装置



- 核聚变的关键技术：控制等离子体的形状
- 为什么要控制等离子体的形状：目的是为了发电
 - 不同形状的等离子体具有不同的散热、稳定等属性
 - 如果想发电，需要将热传导出来，传出来的热不能太多也不能太少，太多的话容器融化，太少效率低



1. 强化学习能解决的问题

ChatGPT:

- OpenAI于2022年11月30号发布的对话式大型语言模型
- 其功能包括聊天、撰写文章、编写代码等
- 以下为人类与ChatGPT对话的一个例子:

User help me write a short note to introduce myself to my neighbor

ChatGPT Hi there,

My name is [Your Name] and I just moved in next door. I wanted to reach out and introduce myself. I'm looking forward to getting to know you and the neighborhood. If you ever need anything or just want to chat, please don't hesitate to stop by or give me a call.

Best,
[Your Name]

1. 强化学习能解决的问题

Step 1

Collect demonstration data and train a supervised policy.

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3.5 with supervised learning.



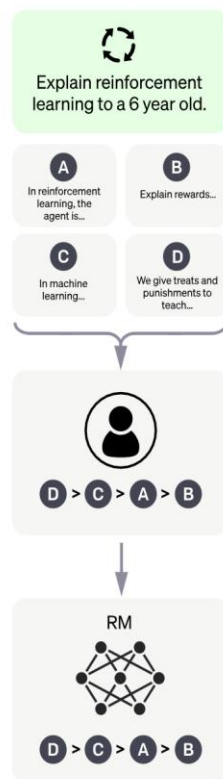
Step 2

Collect comparison data and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using the PPO reinforcement learning algorithm.

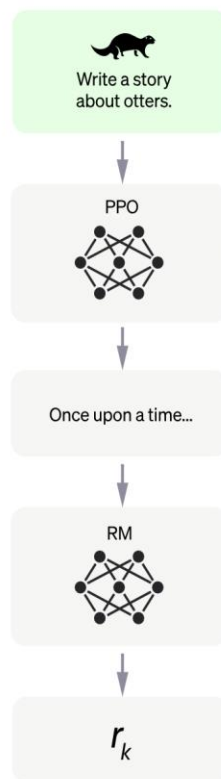
A new prompt is sampled from the dataset.

The PPO model is initialized from the supervised policy.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



左图为ChatGPT的训练思路：

第1步：收集示例数据，得到有监督微调策略SFT

第2步：收集人工标注的对比数据训练**回报模型**

第3步：利用PPO算法和学到的回报模型优化SFT

ChatGPT训练思路



1. 强化学习能解决的问题



DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

DeepSeek-AI

research@deepseek.com

Group Relative Policy Optimization In order to save the training costs of RL, we adopt Group Relative Policy Optimization (GRPO) (Shao et al., 2024), which foregoes the critic model that is typically the same size as the policy model, and estimates the baseline from group scores instead. Specifically, for each question q , GRPO samples a group of outputs $\{o_1, o_2, \dots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model π_{θ} by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)] \frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} A_i, \text{clip} \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1 - \varepsilon, 1 + \varepsilon \right) A_i \right) - \beta \mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) \right), \quad (1)$$

$$\mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) = \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - \log \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - 1, \quad (2)$$

where ε and β are hyper-parameters, and A_i is the advantage, computed using a group of rewards $\{r_1, r_2, \dots, r_G\}$ corresponding to the outputs within each group:

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})}. \quad (3)$$

<https://www.kaggle.com/datasets/hemishveeraboina/aime-problem-set-1983-2024>

Let $f(x) = |x-p| + |x-15| + |x-p-15|$, where $0 < p < 15$. Determine the minimum value taken by $f(x)$

This is an increasing linear function, so the minimum value at $x = p + 15$ is $p + 15$.

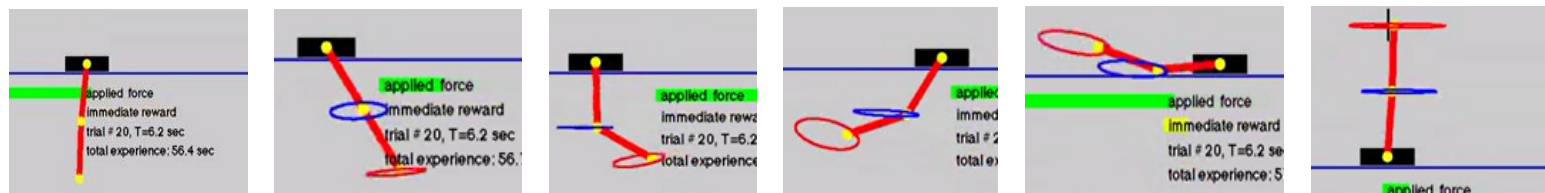
Comparing the minimum values from each interval:

- The value as x approaches p from the left is $-p + 30$.
- The value as x approaches 15 from the left is 15.
- The value at $x = 15$ is 15.
- The value at $x = p + 15$ is $p + 15$.

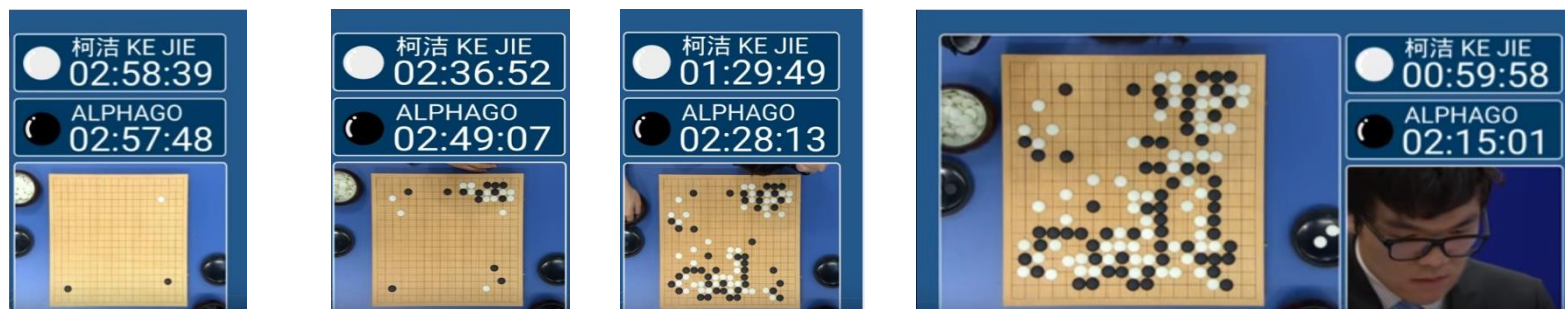
Since $0 < p < 15$, $-p + 30 \geq 15$ and $p + 15 \geq 15$. Therefore, the minimum value of $f(x)$ is 15, achieved at $x = 15$.

Thus, the minimum value taken by $f(x)$ is 15.

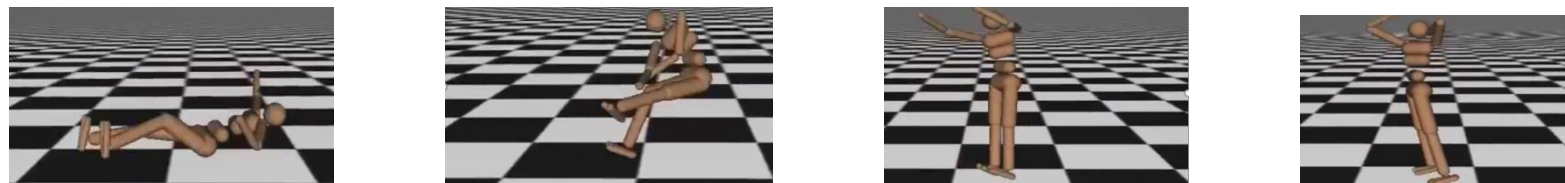
1. 强化学习能解决的问题



图A 非线性系统二级倒立摆

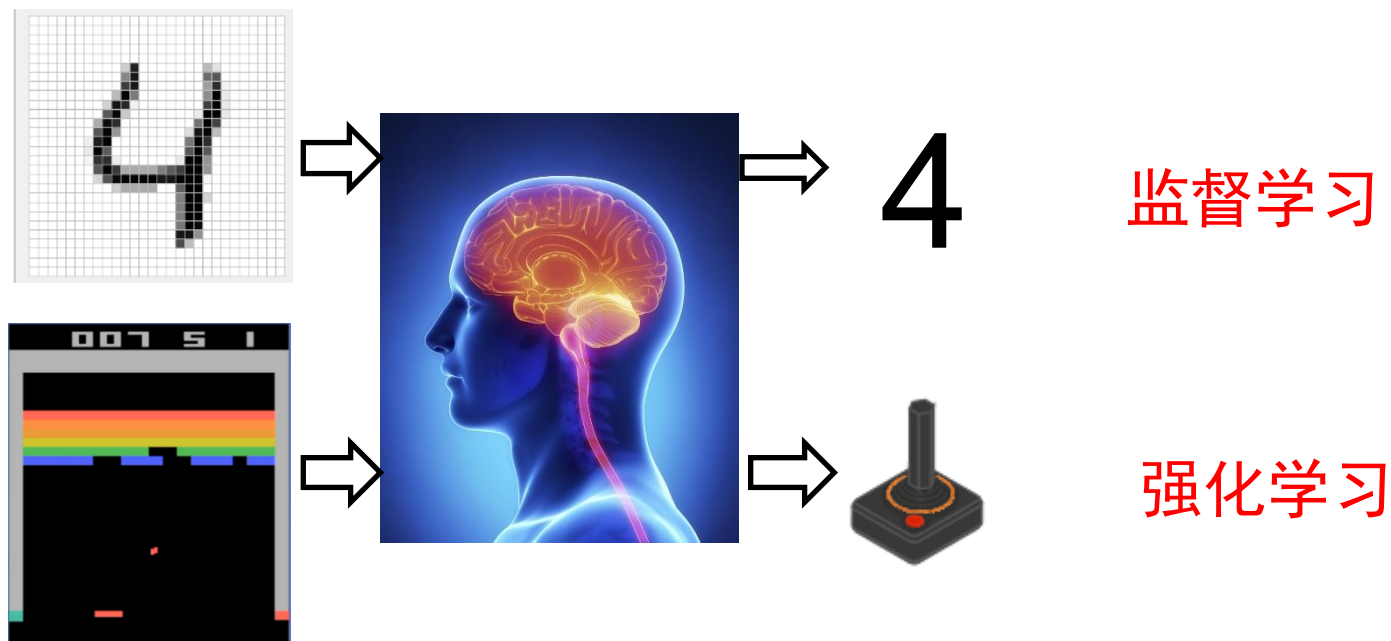


图B AlphaGo与柯洁第二番棋



图C 机器人学习站立

2. 强化学习与其他机器学习的联系和区别



相同点：

都是从数据中学习，逐步改善性能。

不同点：

1. 需要的数据类型不同。监督学习需要标签数据，强化学习需要交互数据。
2. 优化目标不同

2. 强化学习与其他优化方法

序贯最优决策的方法：

当状态空间很小时，可利用线性规划，如资源分配，调度等 $\longrightarrow$ 通过凸优化

当模型已知，回报函数解析时，可用动态规划（最优控制方法） $\longrightarrow$ 解HJB方程

状态方程：

$$\dot{X}=f(t, X, U) \quad X(t_0)=X_0$$

性能指标函数：

$$J[x(t_0), t_0] = \phi[X(t_f), t_f] + \int_{t_0}^{t_f} L(x(t), u(t), t) dt$$

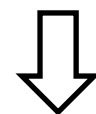
最优控制问题：

$$V(X, t) = \min_{u \in \Omega} \left\{ \phi[X(t_f), t_f] + \int_{t_0}^{t_f} L(x(t), u(t), t) dt \right\}$$

$$V(X, t) = \min_{u \in \Omega} \left\{ \phi[X(t_f), t_f] + \int_{t_0}^{t_f} L(x(t), u(t), t) dt \right\}$$

$$= \min_{u \in \Omega} \left\{ \int_{t_0}^{t_0+dt} L(x(\tau), u(\tau), \tau) d\tau + V(X + \Delta X, t + dt) \right\}$$

$$= \min_{u \in \Omega} \left\{ L(x(t), u(t), t) dt + V(X, t) + \frac{\partial V}{\partial X} f dt + \frac{\partial V}{\partial t} dt + \varepsilon \right\}$$



$$-\frac{\partial V}{\partial t} = \min_{u(t) \in U} \left\{ L(x(t), u(t), t) + \frac{\partial V}{\partial X^T} f[x(t), u(t), t] \right\}$$



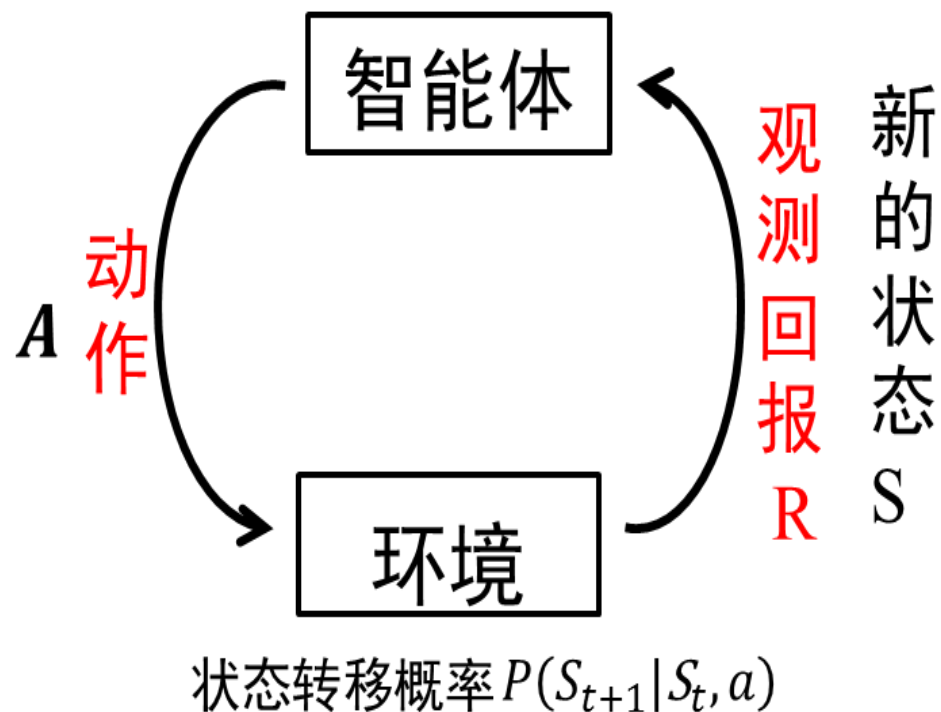
2. 强化学习与其他优化方法

当控制策略简单时，进化算法。————→ 遗传算法

当模型未知或非常复杂，回报函数未知，**强化学习算法**。

强化学习是基于采样（与环境交互）的方法，学习最优策略 —————→ **通过交互的方法**

3. 强化学习如何解决问题



强化学习中的基本概念：

策略(policy)：状态空间到动作空间的映射

$$S \rightarrow A$$

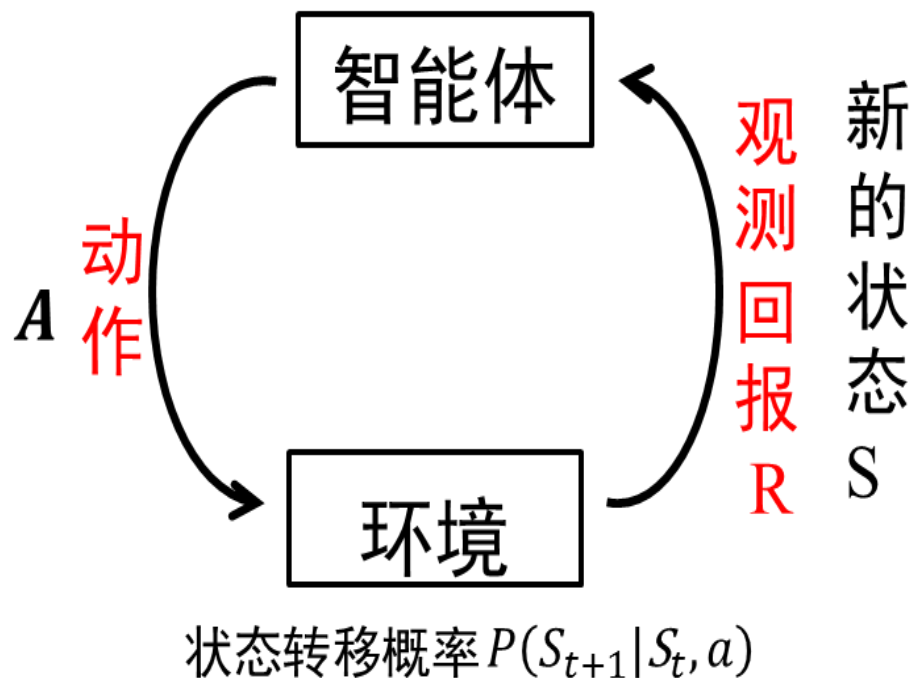
奖赏(reward)：每个时间步，环境给智能体的立即回报

值函数(value function)：从该状态开始，采用该策略所获得的累积回报。

学习目标： $\pi(\cdot | s) : s \mapsto a$

使得长期累积回报（值函数）最大

3. 强化学习如何解决问题



学习目标: $\pi(\cdot | s) : s \mapsto a$

使得长期累积回报（值函数）最大

与环境交互，产生数据流：

$$s_0 \xrightarrow{a_1} (r_1, s_1) \xrightarrow{a_2} (r_2, s_2) \rightarrow \dots$$

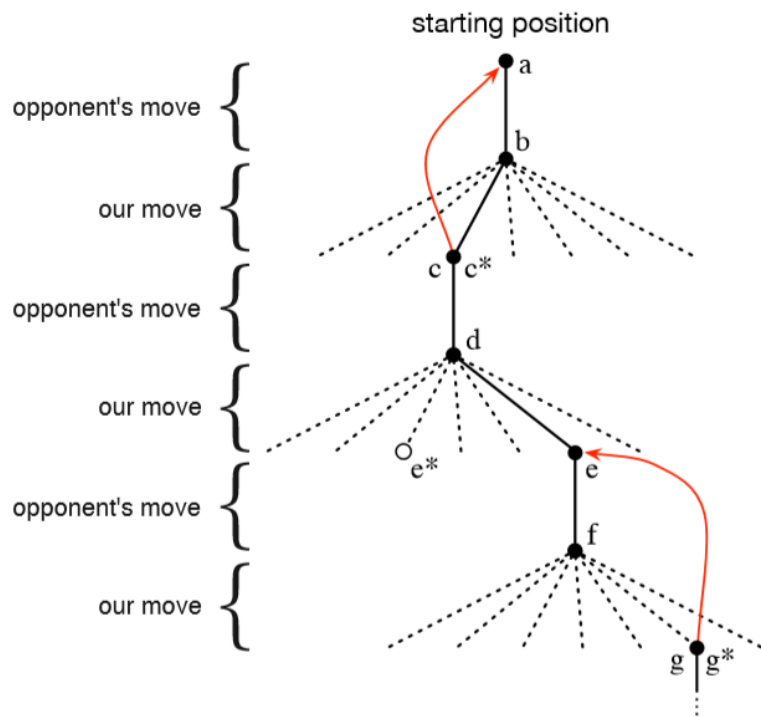
智能体利用数据进行学习，优化自身行为。

强化学习的特征：

1. 试错探索
2. 延迟回报

学习过程可以纳入马尔科夫决策过程的框架中

3. 强化学习如何解决问题



采样过程

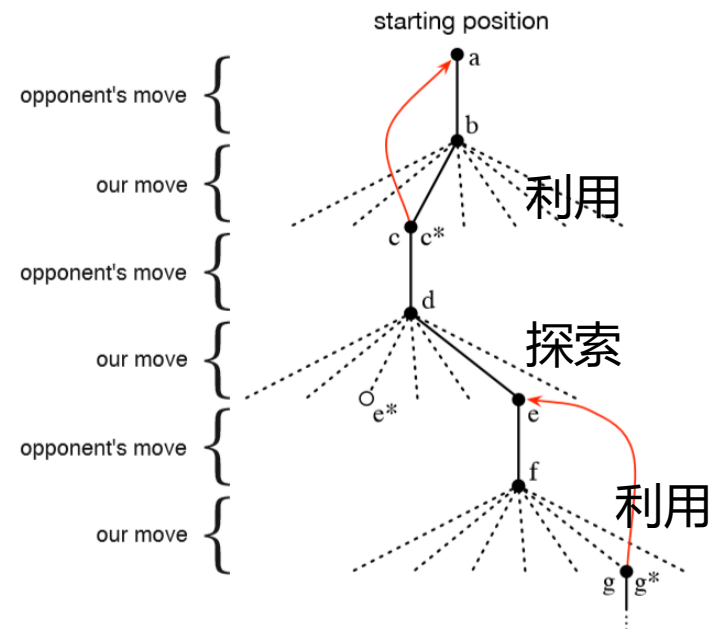
X	O	O
O	X	X
		X

井字游戏

3. 强化学习如何解决问题

- (1) 设置值函数列表
- (2) 每个值表示从当前状态赢的概率
- (3) 赢的状态的赢率为1， 输的状态的赢率为0， 其他状态初始值为0.5

We then play many games against the opponent. To select our moves we examine the states that would result from each of our possible moves (one for each blank space on the board) and look up their current values in the table. Most of the time we move *greedily*, selecting the move that leads to the state with greatest value, that is, with the highest estimated probability of winning. Occasionally, however, we select randomly from among the other moves instead. These are called *exploratory* moves because they cause us to experience states that we might otherwise never see. A sequence of moves made and considered during a game can be diagrammed as in Figure 1.1.



采样过程

Backup:
$$V(s_t) \leftarrow V(s_t) + \alpha[V(s_{t+1}) - V(s_t)]$$



4. 强化学习算法历史

1. 1998年前，基本理论框架形成。

Sutton 的书 《Reinforcement Learning An Introduction》

基于值函数的方法

2. 1998年到2013年，各种直接策略搜索的方法出现

基于策略梯度的RL

基于EM的RL

基于路径积分（Path Integral）的RL

基于回归的RL

基于模型的RL

如何利用回报函数 r
优化最优策略

3. 2013年到现在，深度强化学习火热

深度学习强大的表示能力+强化学习强大的决策能力= 强强联合超越人类

DQN, DNC, AlphaGo等



5. 强化学习的分类

根据是否依赖模型分为：**基于模型**的强化学习和**无模型**的强化学习

基于模型的强化学习算法：用数据先学习系统模型，然后基于模型得到最优策略

无模型的强化学习算法：直接通过交互数据得到最优策略

根据策略更新方法：**基于值函数**的强化学习，**基于直接策略搜索**的强化学习，**Actor-Critic**的方法。

基于值函数的方法：求出最优值函数，然后重构出最优策略

基于直接策略搜索的方法：直接在策略空间进行搜索

Actor-Critic方法：同时逼近值函数和最优策略，类似于策略迭代



5. 强化学习的分类

根据回报函数是否已知分为：正向强化学习和逆向强化学习

正向强化学习：从回报学到最优策略

逆向强化学习：从专家示例中学到回报函数

根据任务大小和多少分为：分层强化学习、元强化学习、多智能体强化学习、迁移学习等



6. 强化学习的发展趋势

1. 强化学习算法与深度学习结合会更加紧密。

DQN的成功离不开CNN—值函数表示

Alpha Zero的成功离不开CNN—策略网络表示

2. 强化学习算法与专业知识结合得将更加紧密。

Alpha Zero 的成功离不开围棋中发展很好的蒙特卡罗树搜索

3. 强化学习算法理论分析会更强，算法会更稳定和高效。

新的探索策略，数据效率的提升。来自统计学的理论知识将会持续发力

4. 强化学习算法跟脑科学、认知神经科学和记忆会更紧密。

大脑的记忆机理，模型和无模型联合机理的利用将飞速提升AI智能



6. 强化学习的发展趋势

1. 贝叶斯强化学习

融合推理能力，可解决POMDP问题

2. 分层强化学习

解决大规模学习问题

3. 元强化学习，多目标强化学习，多任务强化学习

解决多任务学习

4. 多智能体强化学习

博弈：合作，竞争，混合



7. 强化学习课程的路线图

1. 搞清楚马尔科夫决策过程的概念



2. 抓住强化学习的基本迭代过程：策略评估和策略改善



3. 掌握强化学习最常用的两种方法：基于值函数的方法和基于直接策略搜索的方法



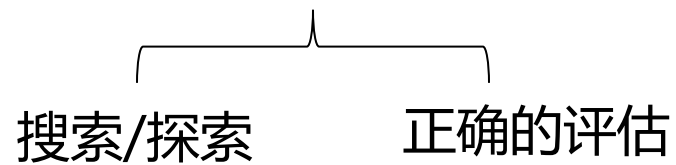
4. 强化学习的其他方法：AC框架，基于模型的强化学习，基于记忆的强化学习等等



强化学习的特点：

利用经验不断学习

更确切地说是利用对当前动作的评估进行学习





Python基础



1. print 语句: print()

```
print('hello reinforcement learning')
```

1.1 打印字符串: %s

```
print('My name is %s'%( 'bao zi xian er'))
```

1.2 打印整数: %d

```
print("I'm %d years old"%(31))
```

1.3 打印浮点数: %f

```
print("I'm %f meters in height"%(1.75))
```

1.4 打印浮点数,并保留两位有效数字: %f

```
print("I'm %.2f meters in height"%1.75)
```

1.5 当然也可以打印中文,中英混合

```
print( "老师how萌傻! ")
```

2. 条件语句: if...else...

```
score =700
```

```
if score >700:
```

```
    print("上清华或北大! ")
```

```
else:
```

```
    print("复读")
```

```
score = 600
```

```
if score >700:
```

```
    print("上清华")
```

```
elif score>=650:
```

```
    print("上其他双一流大学")
```

```
elif score > 600 or score==600:
```

```
    print("上一本")
```

```
else:
```

```
    print("复读")
```



Python基础



3. 循环语句:

3.1. for... in , 依次将list或tuple中的每个元素迭代出来

```
a=[1, 3, 5, 7, 9]
for i in a:
    if i==1:
        print("10以内的奇数为\n%d"%i)
    else:
        print(i)
```

更多例子:

```
b=["天", "地", "玄", "黄"]
for i in b:
    print(i)
for i in range(100):
    print(i)
```

3.2. While循环, 只要条件满足, 就一直循环下去

```
i=0
while i<100:
    print(i)
    i+=1
```

continue和break应用

```
while i<100:
    if i<50:
        i+=1
        continue
    print(i)
    i+=1
    if i>80:
        break
```

跳出本次循环, 不往下继续执行

结束大循环



Python基础



4. 函数定义

利用 `def fun(x)` 定义函数，其中`fun`为定义的函数名，`x`为参数名。

实例：

```
def step(s, a):  
    s_next = s+a*0.01  
    return s_next  
if __name__=="__main__":  
    print(step(2, 3))
```

5. 类，面向对象，对象为程序的基本单元。类包括成员变量和成员函数

```
class maze:  
    def __init__(self, dt):  
        #成员变量用self  
        self.dt = dt  
        #成员函数  
    def step(self, s, a):  
        s_next = s+a*self.dt  
        return s_next  
if __name__=="__main__":  
    maze1=maze(dt=0.01)  
    s_next=maze1.step(2, 3)  
    print(s_next)
```



Numpy 基础



1. 创建矩阵

```
import numpy as np
from numpy import *
A = np.array([[1, 2, 3], [2, 4, 6]])
```

2. numpy创建的矩阵的属性

维数: ndim

```
print("The dimision of A is %d"%A.ndim)
```

形状: shape

```
print("The shape of A is", A.shape)
```

大小: size

```
print("The size of A is %d"%A.size)
```

3. 创建数组

```
B = np.array([1, 2, 3])
```

将数组变为矩阵（加一个维度）：

```
B=B[np.newaxis, :]
```

4. 创建全零矩阵

```
C = np.zeros((3, 3))
print(C)
```

5. 利用np.arange函数创建整数数组

```
D = np.arange(10)
print(D)
```

6. 利用reshape函数改变数据的形状

```
E= D.reshape(2, 5)
print(E)
```



Numpy 基础

6. 矩阵乘法:

对应元素相乘: `print (A*B)`

矩阵相乘: `print (np. dot (A, b))`

7. 矩阵转置:

`print (np. transpose (A))`

8. 矩阵元素的访问

`print (A[1, 1])`

`print (a1[0, :])`

`print (a1[0, 1:2])`

`print (a1[0, -1])`

9. 矩阵的合并

行合并: `print (np. hstack ((a1, a2)))`

列合并: `print (np. vstack ((a1, a2)))`

10. 常用的函数: sin, cos, exp()

`print (np. sin (1))`

`print (np. cos (1))`

`print (np. exp (1))`



本次作业



1. 阅读sutton书的第一章，并做读书笔记